

**21ECC301TP –MICROPROCESSOR, MICROCONTROLLER
AND INTERFACING TECHNIQUES**

A Laboratory Record

Submitted by

KAVINBHARATH SHANMUGAM PANDIAN - RA2411004040042

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING



**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING FACULTY OF ENGINEERING AND
TECHNOLOGY**

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
VADAPALANI CAMPUS, CHENNAI – 26**

SEPTEMBER 2026



BONAFIDE CERTIFICATE

Register No: **RA2411004040042**

Date: **07-10-2026**

Certified that this is the bonafide record of work done by **KAVINBHARATH SHANMUGAM PANDIAN**, of **III Year, B.Tech Electronics and Communication Engineering** Degree course who carried out the laboratory work of **21ECC301P – Microprocessor, Microcontroller and Interfacing techniques** in SRM Institute of Science Technology, Vadapalani Campus during the odd semester of the academic year 2026-2027.

Faculty-in-Charge

Date: **07-10-2026**

Head of the Department

INDEX

S.NO	EXPERIMENT	PAGE
1	Arithmetic Operations	4
2	Stack operations	8
3	Addition of 32-bit numbers	10
4	Addition of 16-bit numbers	12
5	Multiplication of 16-bit numbers	14
6	Subtraction of 16-bit numbers	16
7	Odd and Even number finder	18
8	Largest number program	20
9	Number of 1's in a number	22
10	Scanning a byte of data	24
11	Sorting an Array	26
12	String Operations	28

Aim:-

To write an 8086 ALP to perform basic arithmetic operations using emu8086

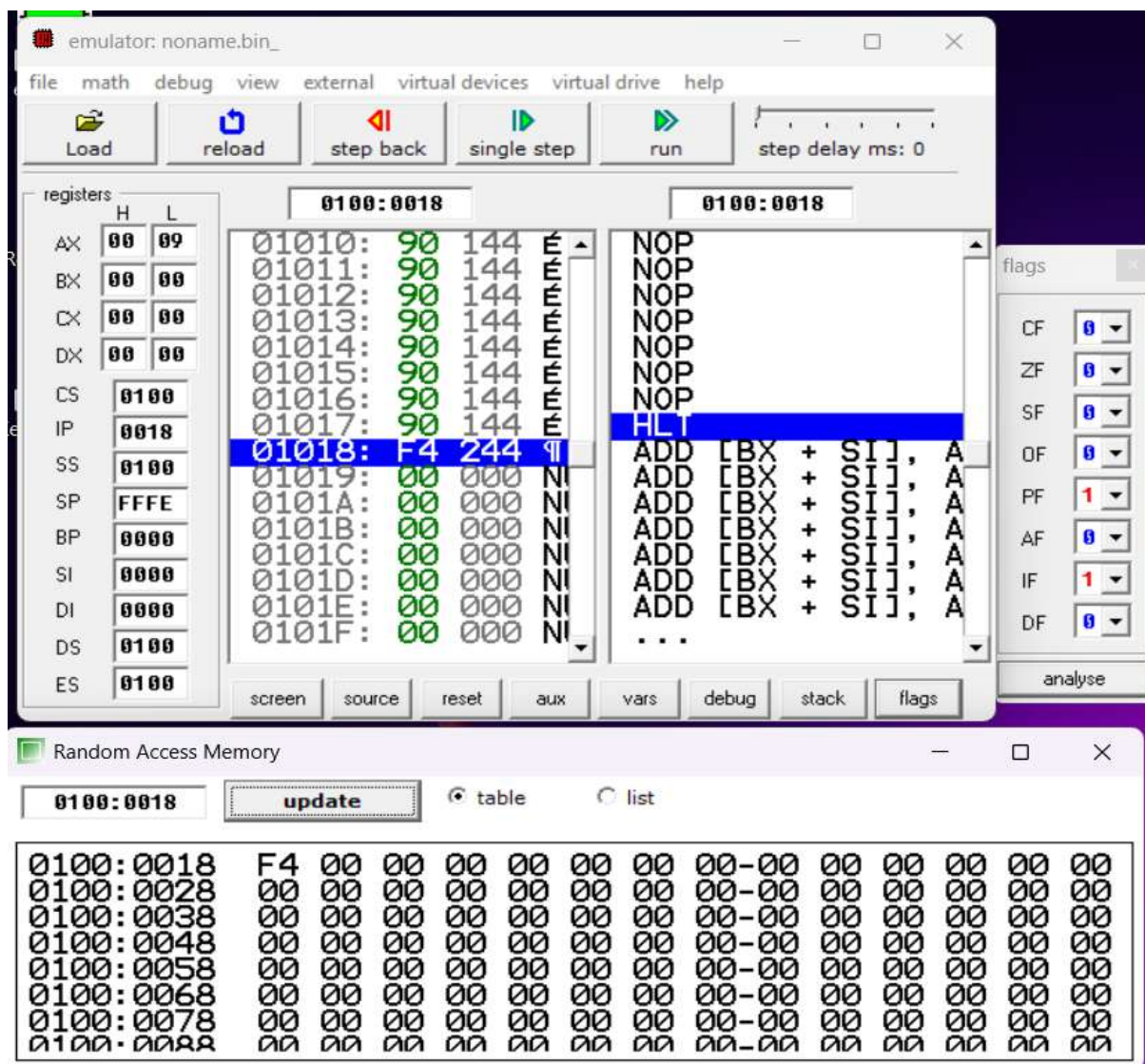
Software required:-

emu8086

Program:-**1) Addition:-**

mov AL, 6H

add AL, 3H

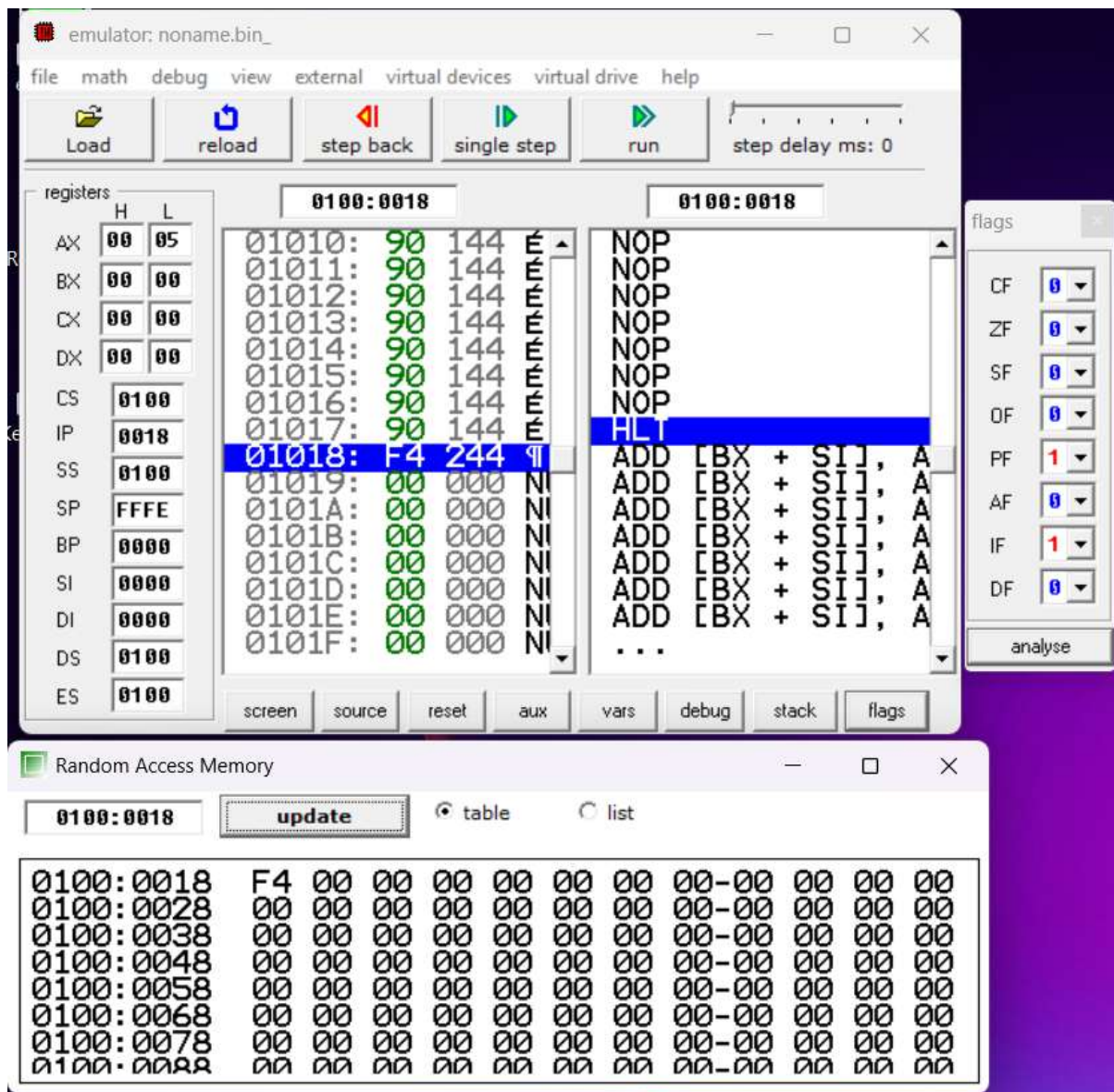


2) Subtraction:-

mov AL, 8H

sub AL, 3H

hlt



emulator: noname.bin_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers

	H	L
AX	00	05
BX	00	00
CX	00	00
DX	00	00
CS	0100	
IP	0018	
SS	0100	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0100	
ES	0100	

0100:0018

Address	Hex	Dec	Op
01010:	90	144	E
01011:	90	144	E
01012:	90	144	E
01013:	90	144	E
01014:	90	144	E
01015:	90	144	E
01016:	90	144	E
01017:	90	144	E
01018:	F4	244	HLT
01019:	00	000	NI
0101A:	00	000	NI
0101B:	00	000	NI
0101C:	00	000	NI
0101D:	00	000	NI
0101E:	00	000	NI
0101F:	00	000	NI

0100:0018

NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
HLT
ADD [BX + SI], A
ADD [BX + SI], A
ADD [BX + SI], A
ADD [BX + SI], A
ADD [BX + SI], A
ADD [BX + SI], A
...

flags

CF 0
ZF 0
SF 0
OF 0
PF 1
AF 0
IF 1
DF 0

analyse

screen source reset aux vars debug stack flags

Random Access Memory

0100:0018 update table list

Address	Hex	Dec	Op
0100:0018	F4 00 00 00 00 00 00 00	00-00 00 00 00	
0100:0028	00 00 00 00 00 00 00 00	00-00 00 00 00	
0100:0038	00 00 00 00 00 00 00 00	00-00 00 00 00	
0100:0048	00 00 00 00 00 00 00 00	00-00 00 00 00	
0100:0058	00 00 00 00 00 00 00 00	00-00 00 00 00	
0100:0068	00 00 00 00 00 00 00 00	00-00 00 00 00	
0100:0078	00 00 00 00 00 00 00 00	00-00 00 00 00	
0100:0088	00 00 00 00 00 00 00 00	00-00 00 00 00	

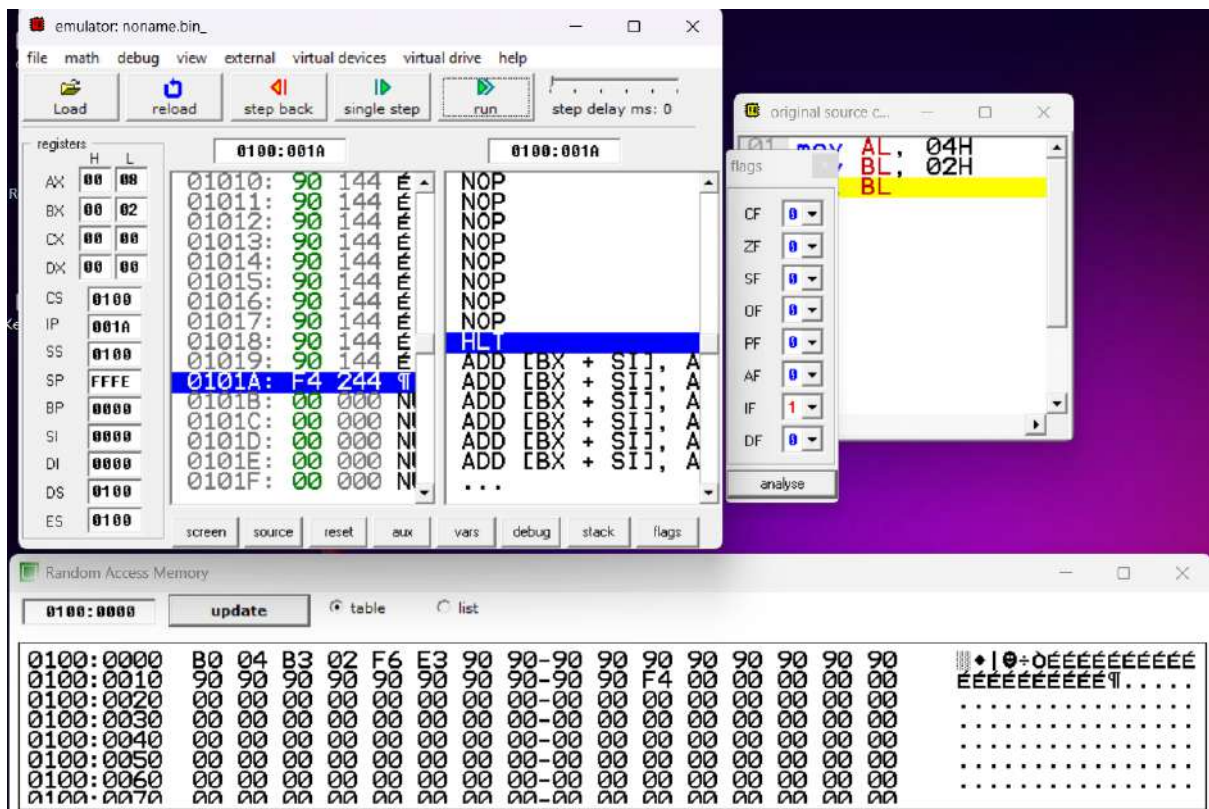
3) Multiplication:-

```
mov AL, 04H
```

```
mov BL, 02H
```

mul BL

hlt



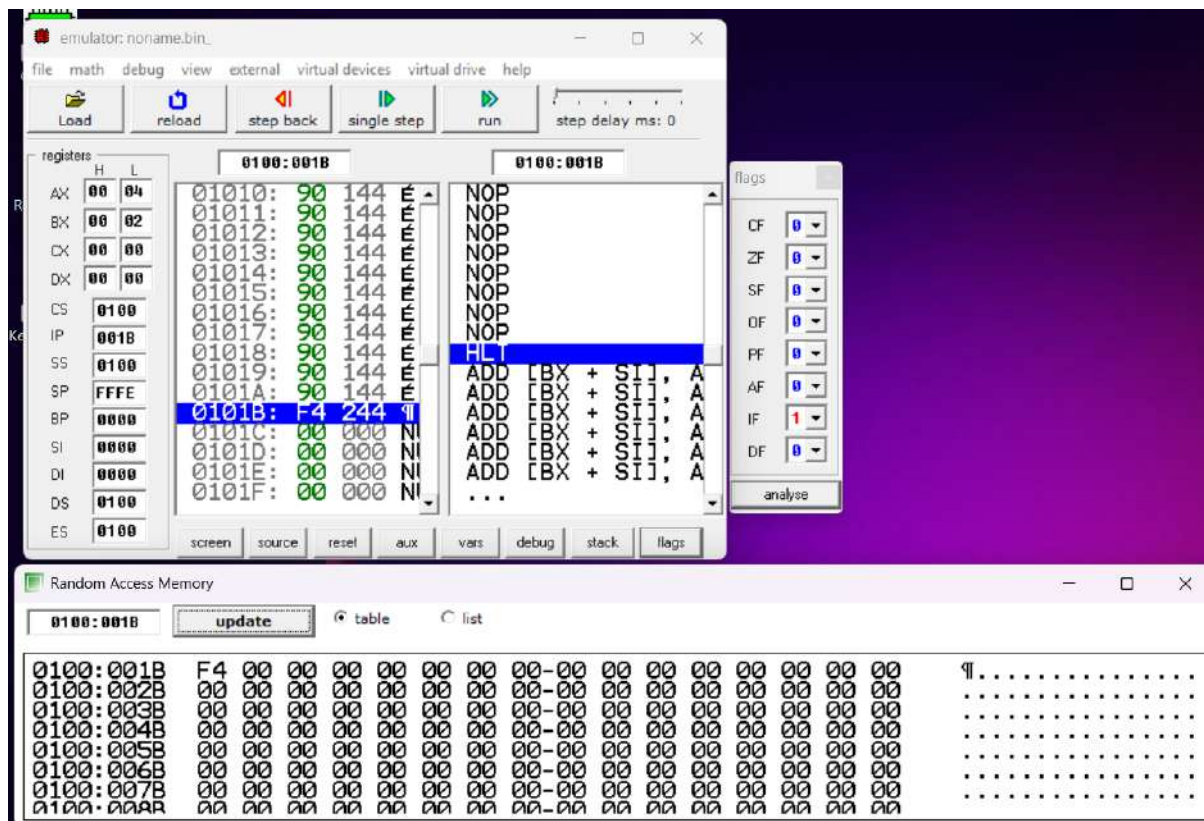
4) Division:-

```
mov AX, 0008H
```

```
mov BL, 02H
```

div BL

hlt



Result:-

Thus an 8086 ALP for arithmetic operations (addition, subtraction, multiplication, and division) using the specified instruction formats has been successfully implemented and its memory output has been verified using emu8086.

Aim: -

To transfer content from AX register to BX register without using the MOV instruction.

Software required: -

emu8086

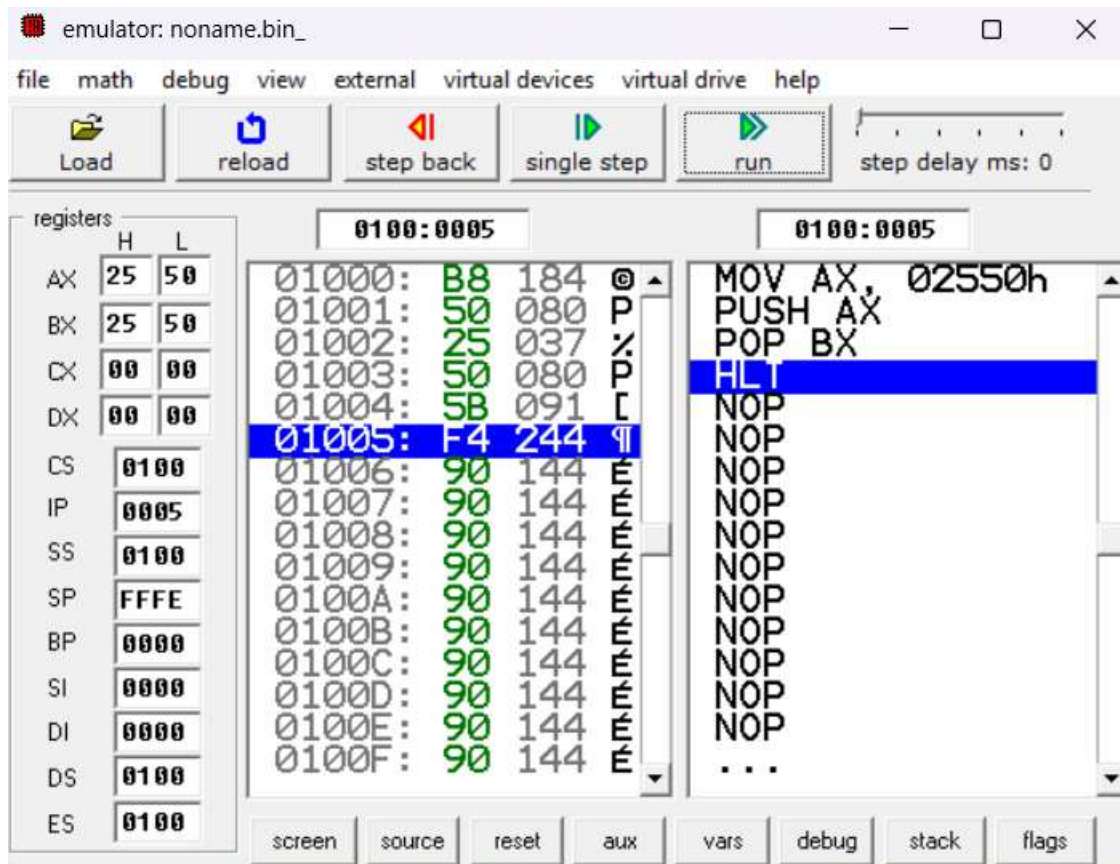
Program: -

```
mov AX, 2550h
```

```
push AX
```

```
pop BX
```

```
hlt
```

Result :-

Thus an 8086 ALP for transferring content from AX to BX using stack operations (PUSH and POP) without mov has been successfully implemented and verified using emu8086.

Exp.no- 3	Addition of 32-bit numbers
-----------	----------------------------

Aim: -

To perform addition of 32-bit number using an 8086 ALP.

Software required: -

emu8086

Program: -

```
mov ax, 2444h
```

```
mov bx, 1111h
```

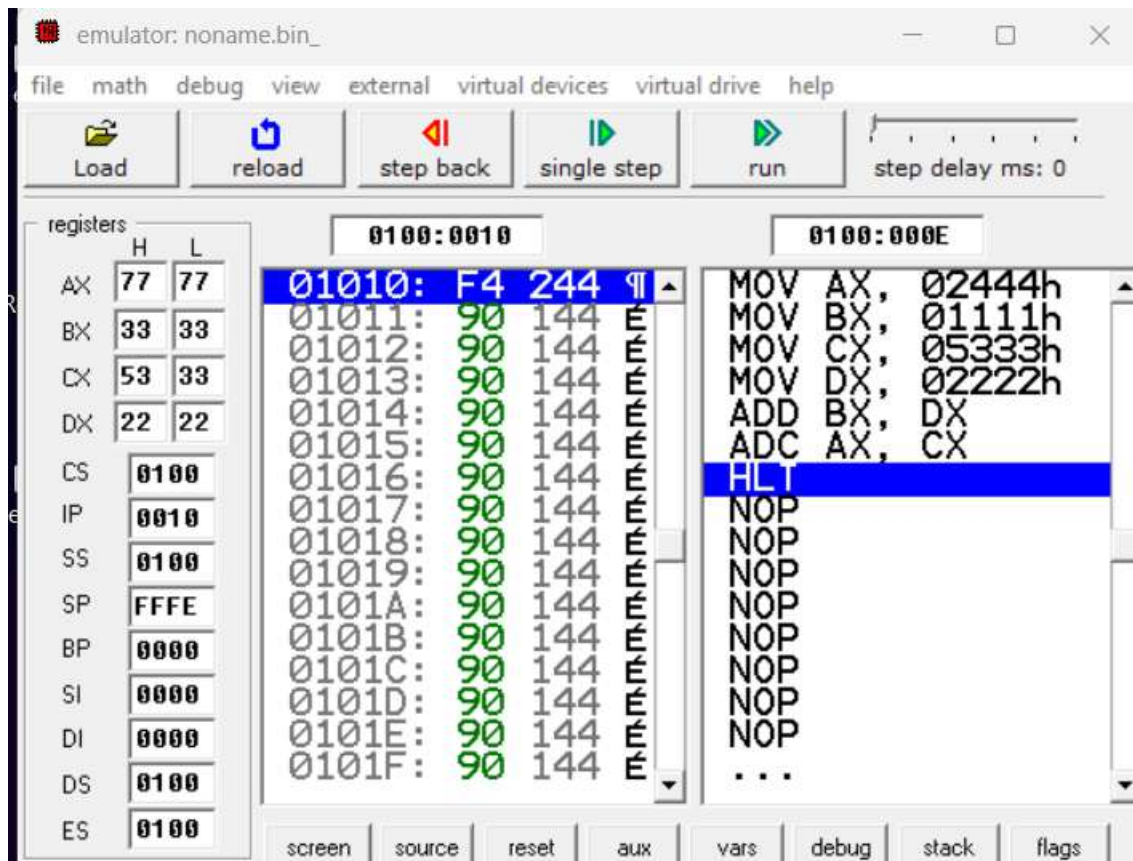
```
mov cx, 5333h
```

```
mov dx, 2222h
```

```
add bx, dx
```

```
adc ax, cx
```

```
hlt
```



Result :-

Thus an 8086 ALP for adding two 32-bit numbers (2444:1111H and 5333:2222H) utilizing ADD and ADC has been successfully implemented and verified using emu8086.

Exp.no- 4	Addition of 16-bit numbers
-----------	----------------------------

Aim: -

To perform addition of two 16-bit numbers and store the answer from 6000h onwards.

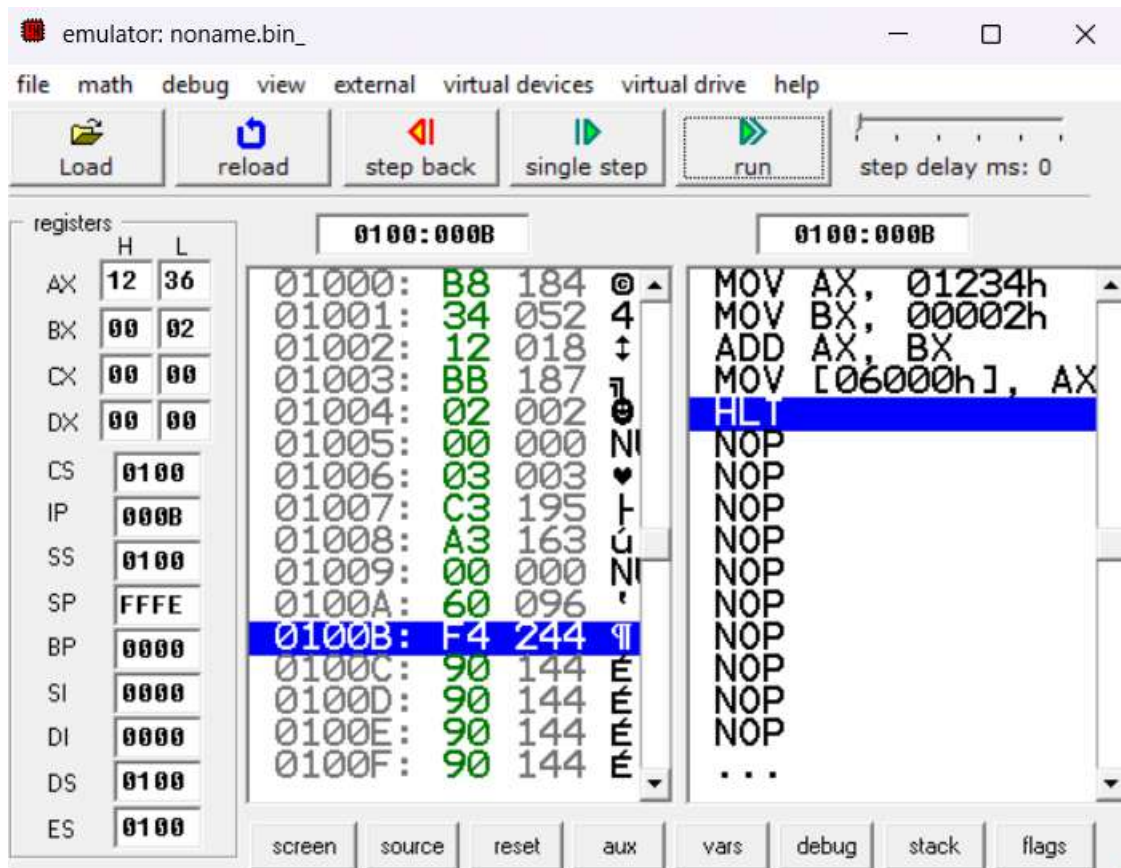
Software required: -

emu8086

Program: -

```
mov ax, 1234h
mov bx, 0002h
add ax, bx
mov [6000h], ax

hlt
```



Result :-

Thus an 8086 ALP for adding 16-bit numbers (1234H and 0002H) and storing the result at memory location 6000H has been implemented and verified using emu8086.

Exp.no- 5	Multiplication of 16-bit numbers
-----------	----------------------------------

Aim: -

To perform multiplication of two 16-bit numbers using an 8086 ALP.

Software required: -

emu8086

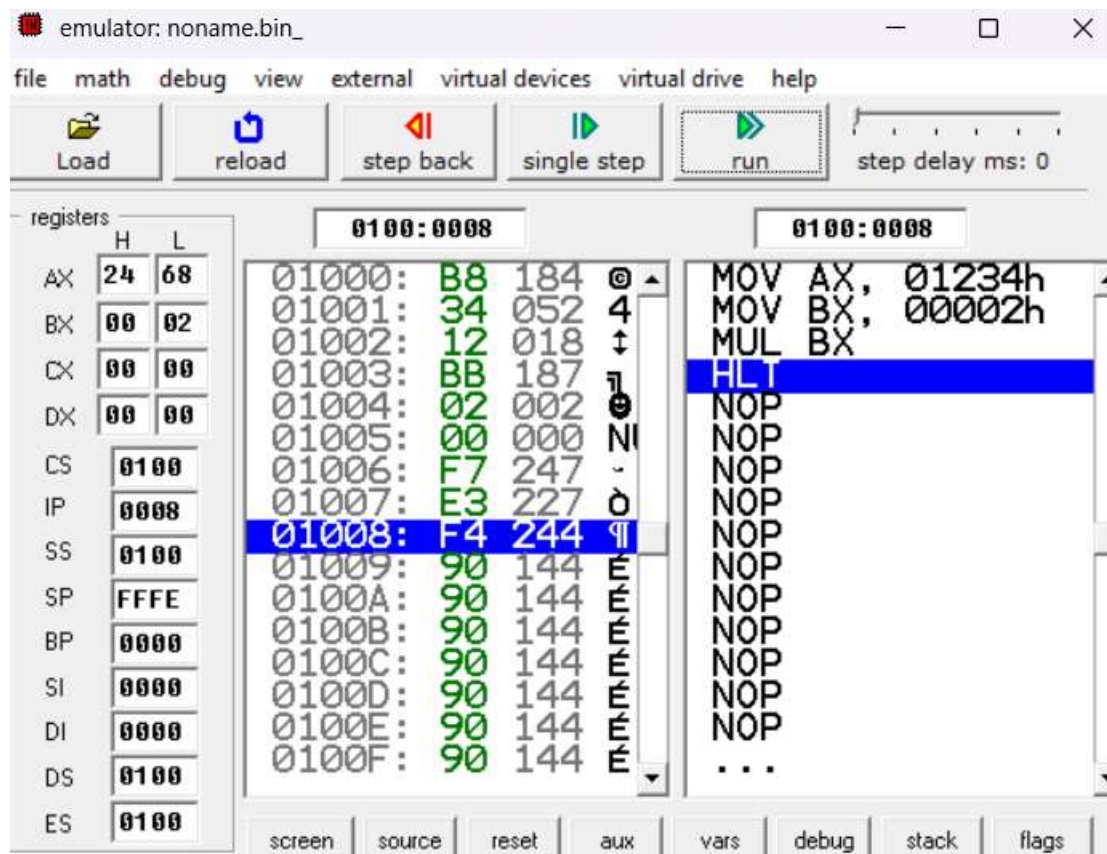
Program: -

```
mov ax, 1234h
```

```
mov bx, 0002h
```

```
mul bx
```

```
hlt
```

Result :-

Thus an 8086 ALP for multiplying two 16-bit numbers (1234H and 0002H) using the MUL instruction has been successfully implemented and verified in emu8086.

Exp.no- 6	Subtraction of 16-bit numbers
-----------	-------------------------------

Aim: -

To perform subtraction of two 16-bit numbers using an 8086 ALP.

Software required: -

emu8086

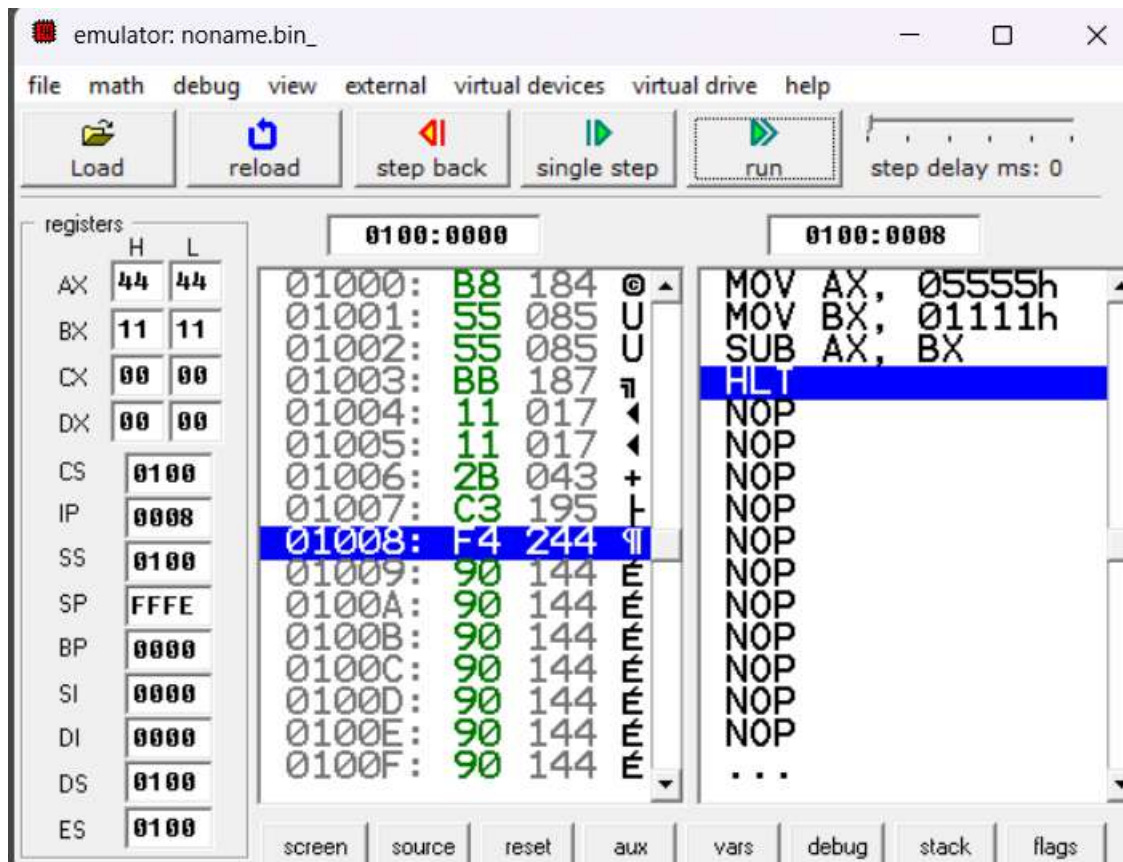
Program: -

```
mov ax, 5555h
```

```
mov bx, 1111h
```

```
sub ax, bx
```

```
hlt
```



Result:-

Thus an 8086 ALP for subtracting two 16-bit numbers (5555H and 1111H) using the SUB instruction has been successfully implemented and verified in emu8086.

Exp.no- 7	Odd and Even number finder
-----------	----------------------------

Aim: -

To write an 8086 ALP to find whether a given number is odd or even and store 01h in 5000h if it is even

Software required: -

emu8086

Program: -

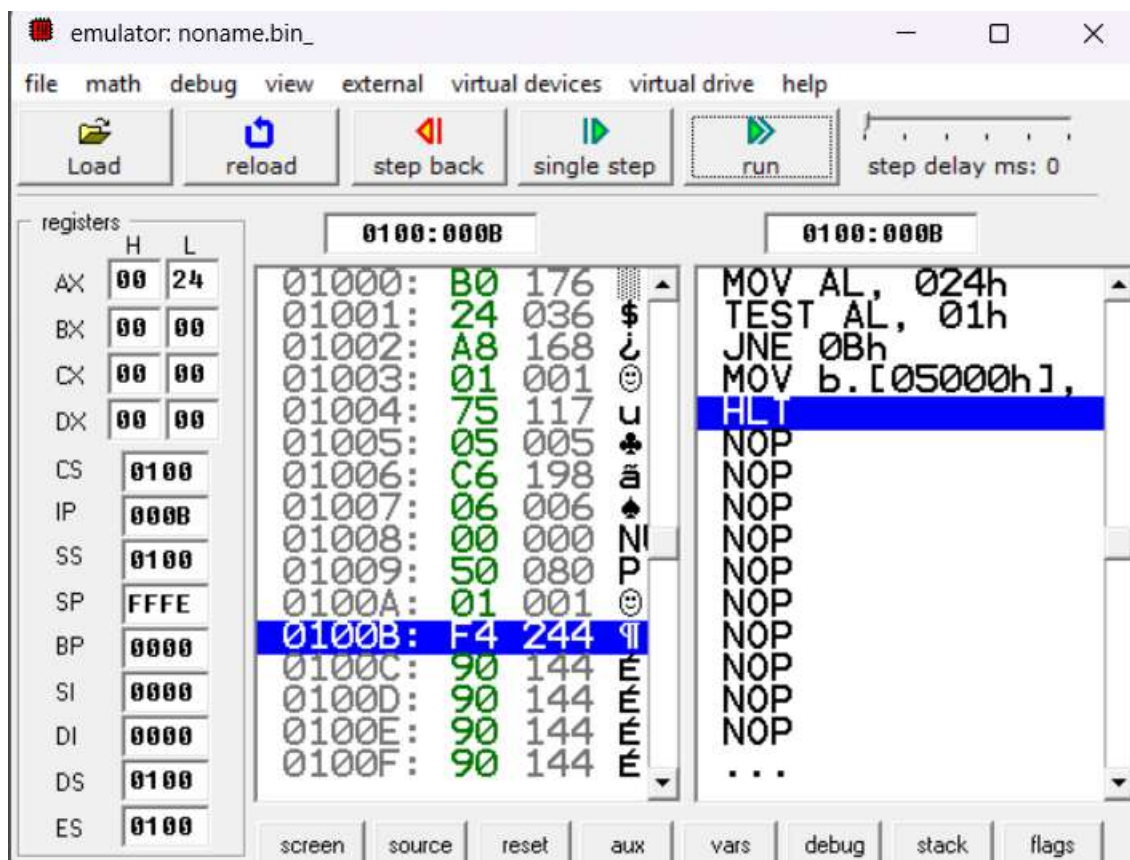
```
mov al, 24h
```

```
test al, 1
```

```
jnz L1
```

```
mov [5000h], 01h
```

```
L1: hlt
```



Result :-

Thus an 8086 ALP for finding if a number (24H) is odd or even using TEST and conditional jump instructions has been implemented and verified in emu8086.

Exp.no- 8	Largest number program
-----------	------------------------

Aim: -

To write an 8086 ALP to find the largest of given numbers.

Software required: -

emu8086

Program: -

```
mov al, 12h
```

```
mov bl, 45h
```

```
mov cl, 23h
```

```
cmp al, bl
```

```
jnc next
```

```
mov al, bl
```

```
next:
```

```
cmp al, cl
```

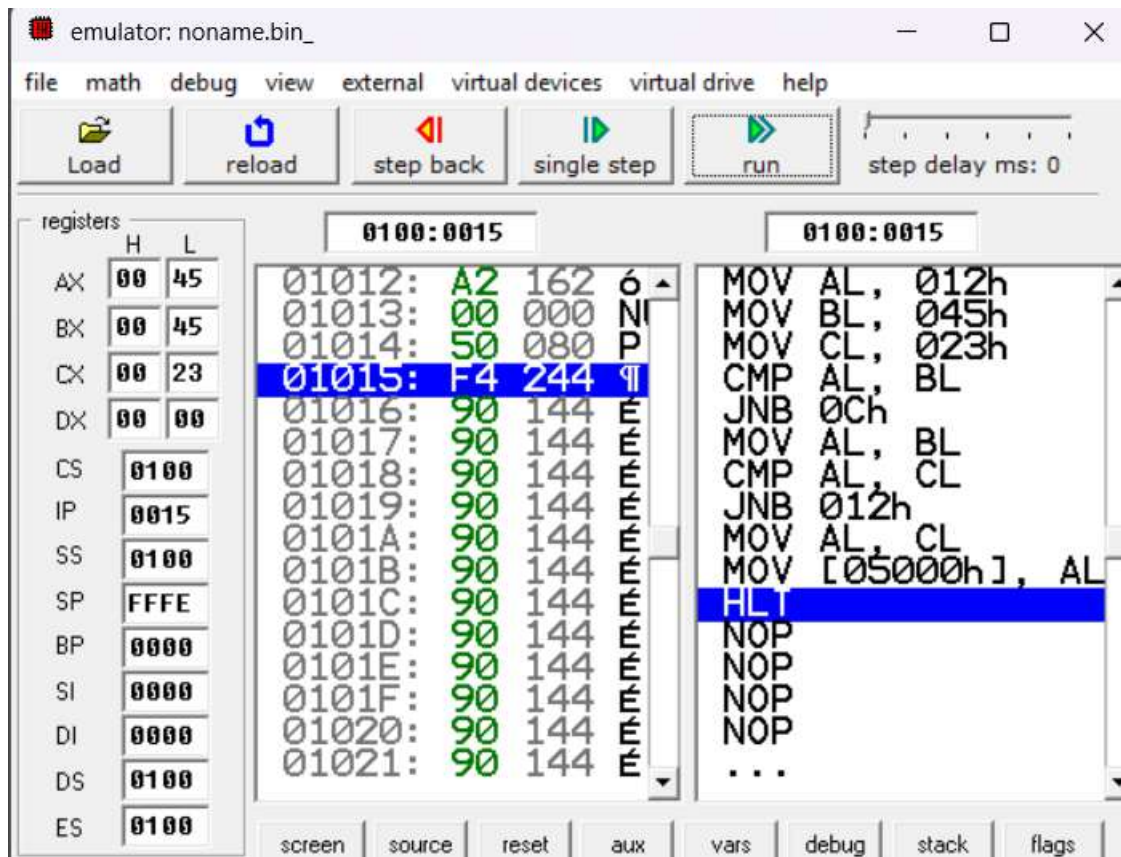
```
jnc store
```

```
mov al, cl
```

store:

mov [5000h], al

hlt



Result:-

Thus an 8086 ALP for finding the largest among given register values (12H, 45H, 23H) and storing the maximum value at 5000H has been implemented and verified using emu8086.

Exp.no- 9	Number of 1's in a number
------------------	----------------------------------

Aim: -

To write an 8086 ALP for calculating the number of 1's in a given number

Software required: -

emu8086

Program: -

```
mov al, 45h
```

```
mov cx, 8
```

```
mov bl, 0
```

```
L1:shr al, 1
```

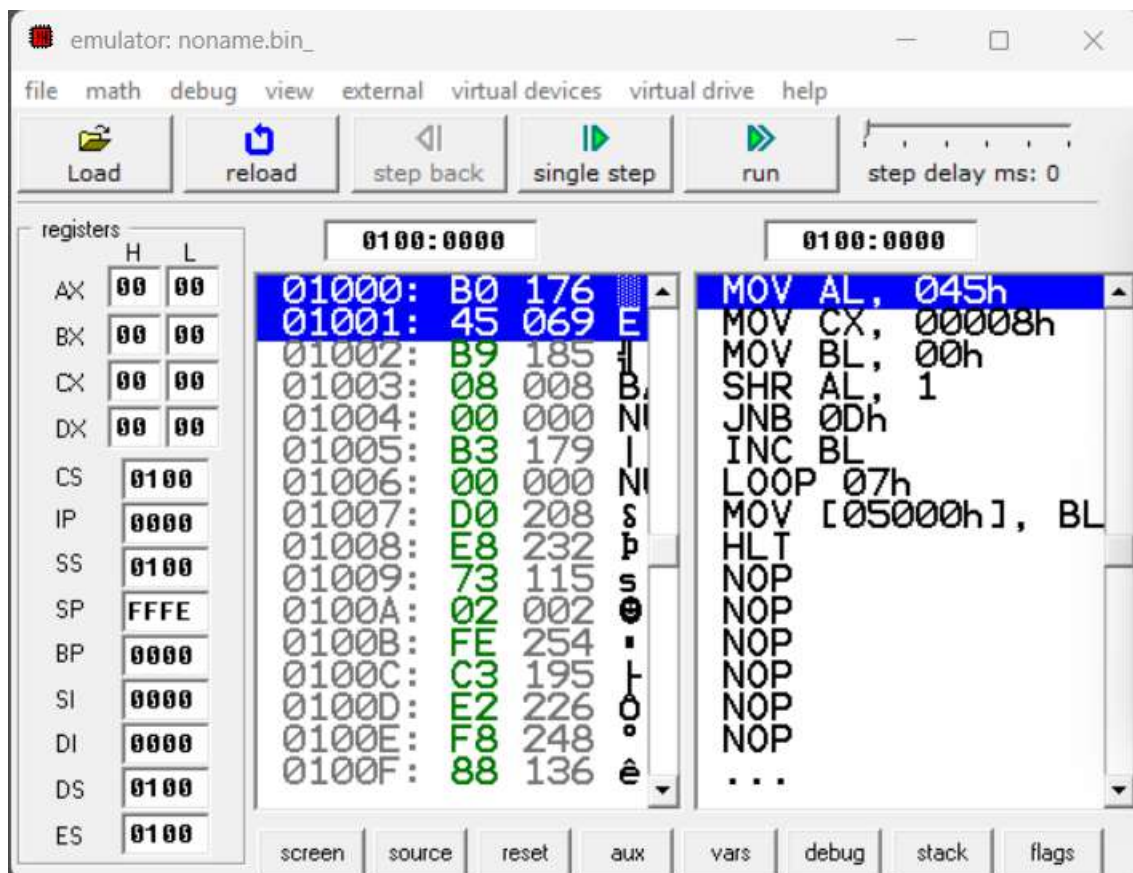
```
    jnc L2
```

```
    inc bl
```

```
L2:loop L1
```

```
mov [5000h], bl
```

```
hlt
```



Result :-

Thus an 8086 ALP for calculating the number of 1's in a given data byte (45H) using right shift (SHR) and loop operations has been implemented and verified using emu8086.

Aim: -

To write an 8086 ALP for scanning a byte of data.

Software required: -

emu8086

Program: -

```
mov si, 4000h
```

```
mov cx, 05h
```

```
mov al, 07h
```

```
L5:cmp al, [si]
```

```
jz L6
```

```
inc si
```

```
dec cx
```

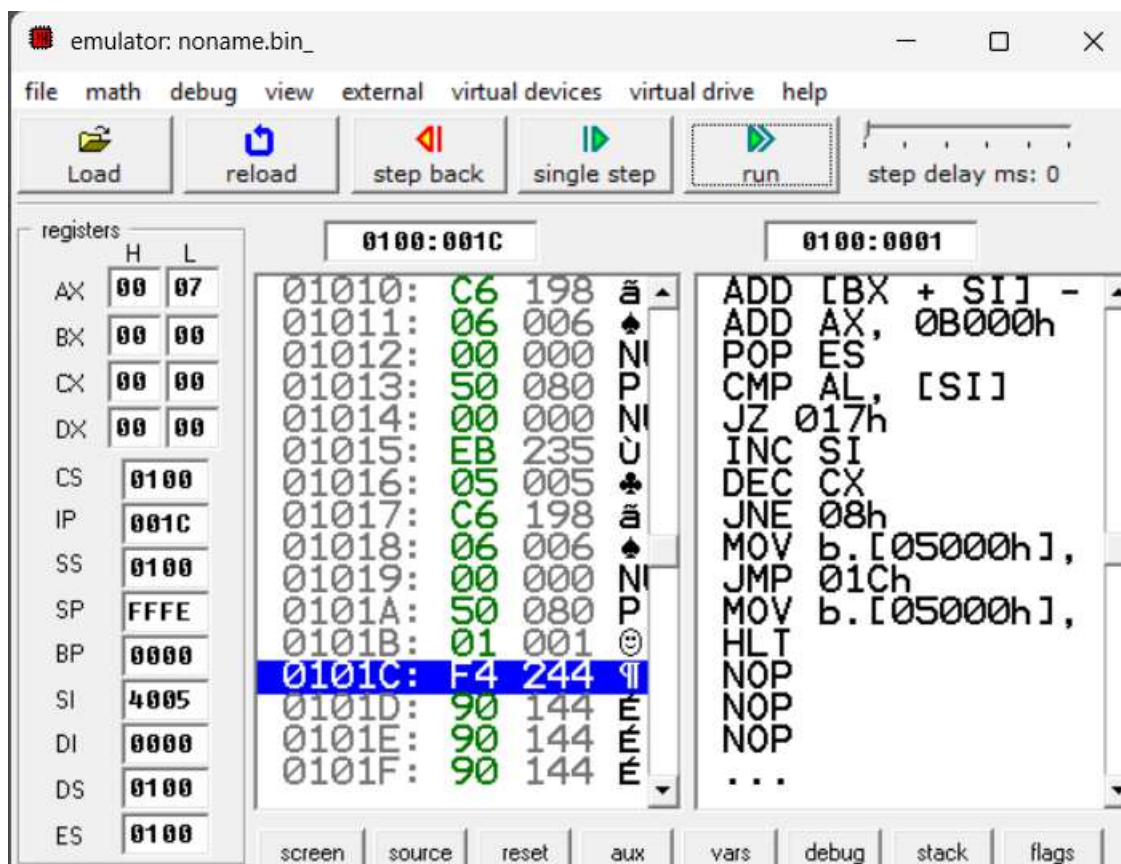
```
jnz L5
```

```
mov [5000h], 00h
```

```
jmp L7
```

```
L6:mov [5000h], 01h
```

L7:hl



Result:-

Thus an 8086 ALP for scanning a specific byte value (07H) across memory locations starting from 4000H and recording the search status at 5000H has been implemented and verified using emu8086.

Aim: -

To write an 8086 ALP for sorting an array.

Software required: -

emu8086

Program: -

mov cl, 03h

dec cl

K7:mov si, 5000h

mov ch, cl

K6:mov al, [si]

mov bl, [si+1]

cmp al, bl

jc K8

mov [si], bl

mov [si+1], al

K8:inc si

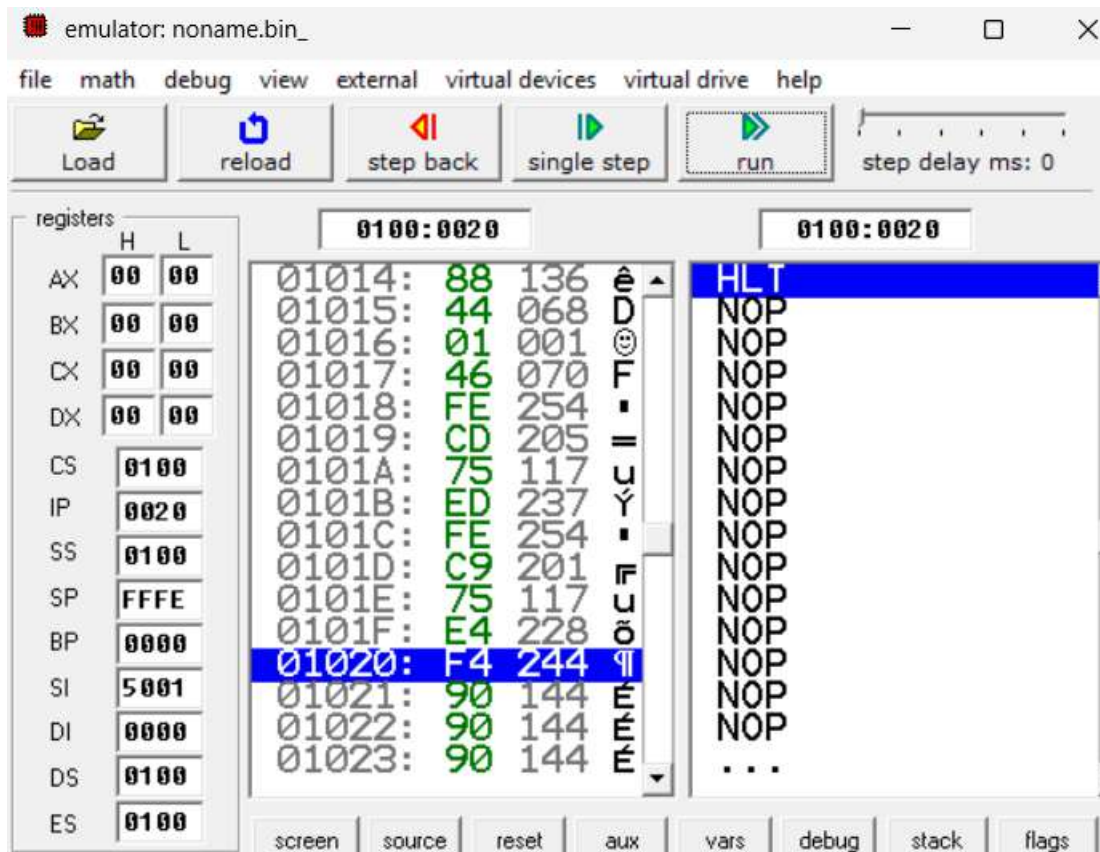
dec ch

jnz K6

dec cl

jnz K7

hlt



Result:-

Thus an 8086 ALP for sorting an array of bytes residing around 5000H in ascending order using a bubble sort logic has been implemented and verified using emu8086.

Aim: -

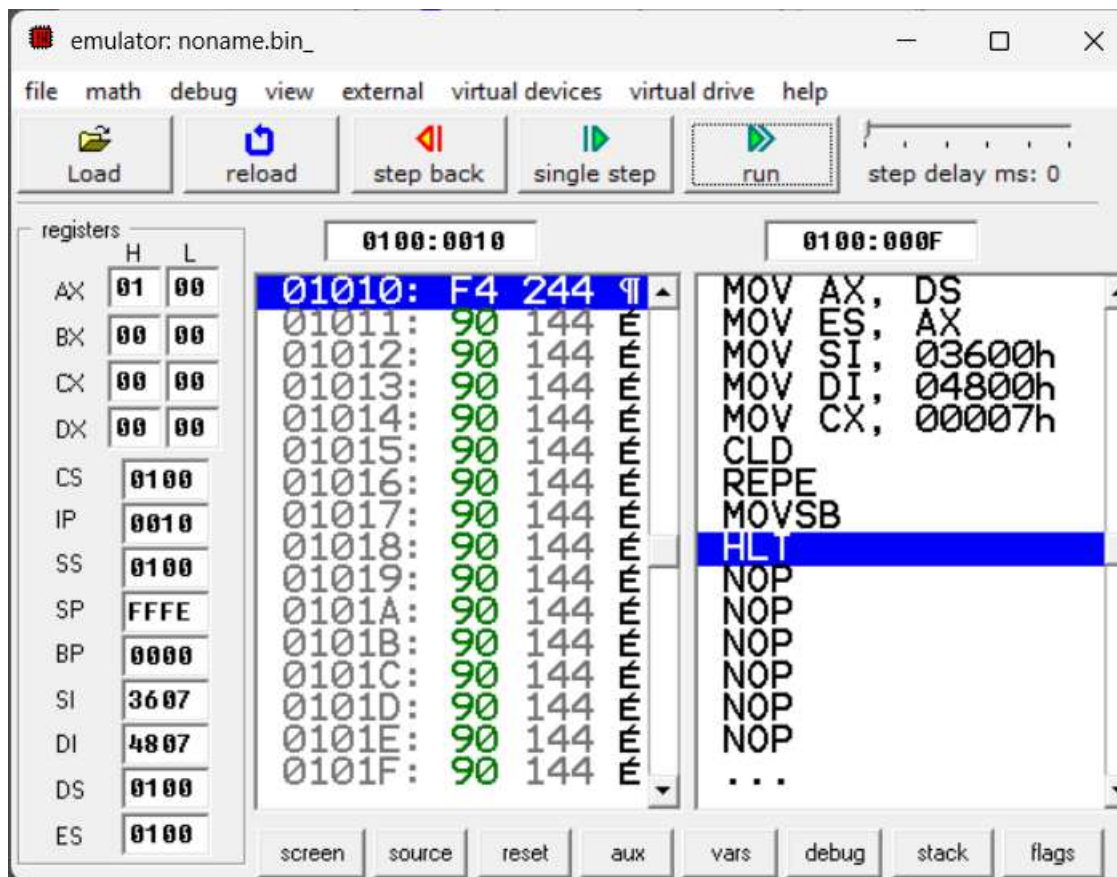
To write an 8086 ALP for demonstrating string operations in emu8086

Software required: -

emu8086

Program: -

```
mov ax, ds
mov es, ax
mov si, 3600h
mov di, 4800h
mov cx, 07h
cld
rep movsb
hlt
```



Result :-

Thus an 8086 ALP for string block transfer operations from source address 3600H to destination address 4800H using REP MOVSB has been implemented and verified using emu8086.